SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA0313	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	G V Durga Charan	**Reg. No.:**	192324100
Section / Batch:	III	**Date:**	04-11-2026

Code of Conduct

I G V Durga Charan (192324100) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: G V Durga Charan

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Matching	Regular Expressions, Basic Regular Expression Patterns	1
Finite State Automata Implementation	Finite State Automata, Models and Algorithms	2
Advanced Regular Expression Search	Regular Expressions, Basic Regular Expression Patterns	3

Annexure A – Coding Challenge

Section 1: Intelligent Email and Password Validator using Regular Expressions

1. **Problem Statement:**

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

Develop a Python application that validates:

Email Address
Password
Mobile Number
using Regular Expressions.

Validation Rules

Email
Must start with an alphabet.
May contain letters, digits, '.', '_' before '@'.
Domain should contain only alphabets.
Valid extensions: .com, .org, .edu, .net, .in

Password

Minimum 8 characters
At least one uppercase letter
At least one lowercase letter
At least one digit
At least one special character (@,#,$,%,&,!)

Mobile Number

Must contain exactly 10 digits.
First digit should be between 6–9.

Expected Output

Display:
Valid Email / Invalid Email
Strong Password / Weak Password
Valid Mobile Number / Invalid Mobile Number

Aim:

To develop a Python program using regular expressions to validate email addresses, passwords, and mobile numbers.

Algorithm:

1. Import Python's re module.
2. Define regular expression patterns for email and mobile number validation.
3. Validate the password using length and character-category checks.
4. Accept user inputs and apply the corresponding patterns.
5. Display the validation result for each input.

Python program:

```
import re
def validate_email(email):
    pattern = r"^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$"
    return re.fullmatch(pattern, email) is not None
def validate_password(password):
    return (
        len(password) >= 8
        and re.search(r"[A-Z]", password)
        and re.search(r"[a-z]", password)
        and re.search(r"\d", password)
        and re.search(r"[@#$%&!]", password)
    )
def validate_mobile(mobile):
    pattern = r"^[6-9]\d{9}$"
    return re.fullmatch(pattern, mobile) is not None
```

```
def main():
    email = input("Enter email address: ").strip()
    password = input("Enter password: ")
    mobile = input("Enter mobile number: ").strip()
    print("\n--- Validation Results ---")
    if validate_email(email):
        print("Valid Email")
    else:
        print("Invalid Email")

    if validate_password(password):
        print("Strong Password")
    else:
        print("Weak Password")
    if validate_mobile(mobile):
        print("Valid Mobile Number")
    else:
        print("Invalid Mobile Number")
if __name__ == "__main__":
    main()
```

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bi
 (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

=============== RESTART: C:/Users/geeth/Downloads/nlp co1at3 1.py =============
Enter email address: abcdefg@gmail.com
Enter password: stud@123
Enter mobile number: 1234567891

--- Validation Results ---
Valid Email
Weak Password
Invalid Mobile Number

=============== RESTART: C:/Users/geeth/Downloads/nlp co1at3 1.py =============
Enter email address: saveetha21@gmail.com
Enter password: Student@145
Enter mobile number: 8123564921

--- Validation Results ---
Valid Email
Strong Password
Valid Mobile Number
```

Explanation

- re.fullmatch() ensures that the entire email or mobile number matches the required pattern.
- [A-Za-z] matches alphabets.
- [A-Za-z0-9._]* allows letters, digits, dots, and underscores before @.
- \.(com|org|edu|net|in) restricts the extension to the five permitted values.
- len(password) >= 8 checks the minimum password length.
- re.search() checks for uppercase, lowercase, digit, and permitted special characters.
- ^[6-9]\d{9}$ ensures that the mobile number contains exactly 10 digits and starts with a digit from 6 to 9.

Section 2: Design a Finite State Automata (FSA) Simulator

2. Problem Statement

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

The program should:

- Accept the DFA description
  - States
  - Input alphabet
  - Transition table
  - Initial state
  - Final state(s)
- Accept multiple input strings.
- Simulate state transitions.
- Display whether the string is Accepted or Rejected.

Example

Language:
Strings ending with "ab"

Input

abaab

Output

Transition Path:
q0 → q1 → q0 → q1 → q2
Accepted

Aim:

To implement a Deterministic Finite Automaton (DFA) simulator that accepts a DFA description, processes multiple strings, displays the transition path, and determines whether each string is accepted or rejected.

Algorithm

1. Read the set of states and input alphabet.
2. Read the initial state and final states.
3. Construct a transition table for every state and input symbol.
4. Accept multiple input strings.
5. Start processing each string from the initial state.
6. Follow the transition table for every symbol.
7. If a transition is undefined, reject the string.
8. After processing all symbols, accept the string if the current state is a final state; otherwise, reject it.

Python program

```
def get_items(prompt):
    return input(prompt).strip().split()
def build_dfa():
    states = get_items("Enter states (space-separated): ")
    alphabet = get_items("Enter input alphabet (space-separated): ")

    if not states or not alphabet:
        raise ValueError("States and alphabet cannot be empty.")

    if len(set(states)) != len(states):
        raise ValueError("States must be unique.")

    if len(set(alphabet)) != len(alphabet):
        raise ValueError("Alphabet symbols must be unique.")

    initial_state = input("Enter initial state: ").strip()

    if initial_state not in states:
```

```
        raise ValueError("Initial state must belong to the state set.")

    final_states = get_items("Enter final states (space-separated): ")

    if not set(final_states).issubset(set(states)):
        raise ValueError("All final states must belong to the state set.")

    transitions = {}

    print("\nEnter the transition table:")
    for state in states:
        transitions[state] = {}

        for symbol in alphabet:
            next_state = input(
                f"Transition from {state} on '{symbol}': "
            ).strip()

            if next_state not in states:
                raise ValueError("Next state must belong to the state set.")

            transitions[state][symbol] = next_state

    return states, alphabet, initial_state, set(final_states), transitions

def simulate_dfa(string, alphabet, initial_state,
                 final_states, transitions):
    current_state = initial_state
    path = [current_state]

    for symbol in string:
        if symbol not in alphabet:
            return False, path, (
                f"Invalid input symbol: {symbol}"
            )

        current_state = transitions[current_state][symbol]
        path.append(current_state)

    accepted = current_state in final_states
    return accepted, path, None

def main():
    try:
        states, alphabet, initial_state, final_states, transitions = (
            build_dfa()
        )

        count = int(input("\nHow many strings do you want to test? "))

        if count < 1:
            print("Enter at least one string.")
            return

        for i in range(count):
            string = input(f"\nEnter string {i + 1}: ")

            accepted, path, error = simulate_dfa(
                string, alphabet, initial_state,
                final_states, transitions
            )

            print("Transition Path:", " -> ".join(path))
```

```
            if error:
                print(error)
                print("Rejected")
            elif accepted:
                print("Accepted")
            else:
                print("Rejected")

    except ValueError as error:
        print("Input Error:", error)
if __name__ == "__main__":
    main()
```

```
Enter states (space-separated): q0 q1 q2
Enter input alphabet (space-separated): a b
Enter initial state: q0
Enter final states (space-separated): q2

Enter the transition table:
Transition from q0 on 'a': q1
Transition from q0 on 'b': q0
Transition from q1 on 'a': q1
Transition from q1 on 'b': q2
Transition from q2 on 'a': q1
Transition from q2 on 'b': q0

How many strings do you want to test? 3

Enter string 1: aabb
Transition Path: q0 -> q1 -> q1 -> q2 -> q0
Rejected

Enter string 2: abab
Transition Path: q0 -> q1 -> q2 -> q1 -> q2
Accepted

Enter string 3: abaab
Transition Path: q0 -> q1 -> q2 -> q1 -> q1 -> q2
Accepted
```

Explanation

- A DFA consists of a finite set of states, an input alphabet, a transition function, an initial state, and final states.
- transitions[state][symbol] determines the next state.
- path records each visited state, including the initial state.
- A string is accepted only if its final state belongs to the set of final states.
- Invalid symbols are rejected instead of being processed as valid transitions.
- The program tests multiple strings against the same DFA.

Section 3: Smart Pattern Matching Engine using Regular Expressions

3. Problem Statement

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

The system should support:
Word Search
Prefix Search
Suffix Search
Date Search
Phone Number Search
Hashtag Search
Mention (@username) Search

Example Input
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing

User Menu
1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix

Expected Output
Display all matching patterns based on user selection.

Aim:
To develop a Python-based text search engine that performs word, prefix, suffix, date, phone number, hashtag, and mention searches using regular expressions.

Algorithm
1. Accept multiline text from the user.
2. Display a menu of search operations.
3. Read the user's choice and the required search term, if applicable.
4. Select the corresponding regular expression.
5. Use re.findall() or re.finditer() to identify matching patterns.
6. Display all matches and their count.
7. Repeat until the user selects Exit.

Python program

```
import re


def search_pattern(text, pattern):
    return re.findall(pattern, text, flags=re.IGNORECASE)


def main():
    print("Enter the text (type END on a new line to finish):")

    lines = []
    while True:
        line = input()
```

```
    if line == "END":
        break

    lines.append(line)

text = "\n".join(lines)

patterns = {
    "1": (r"\b\d{1,2}/\d{1,2}/\d{4}\b", "Date"),
    "2": (r"(?<!\d)[6-9]\d{9}(?!\d)", "Phone Number"),
    "3": (r"(?<!\w)#[A-Za-z0-9_]+", "Hashtag"),
    "4": (r"(?<!\w)@[A-Za-z0-9_]+", "Mention"),
}

while True:
    print("\n--- Smart Pattern Matching Engine ---")
    print("1. Search Date")
    print("2. Search Phone Number")
    print("3. Search Hashtag")
    print("4. Search Mention")
    print("5. Search Prefix")
    print("6. Search Suffix")
    print("7. Search Word")
    print("8. Exit")

    choice = input("Enter your choice: ").strip()

    if choice == "8":
        print("Exiting the search engine.")
        break

    if choice in patterns:
        pattern, label = patterns[choice]
        matches = search_pattern(text, pattern)

    elif choice == "5":
        prefix = input("Enter prefix: ").strip()

        if not prefix or not re.fullmatch(r"\w+", prefix):
            print("Enter a valid prefix.")
            continue

        pattern = r"\b" + re.escape(prefix) + r"\w*\b"
        label = "Prefix"
        matches = search_pattern(text, pattern)

    elif choice == "6":
        suffix = input("Enter suffix: ").strip()

        if not suffix or not re.fullmatch(r"\w+", suffix):
            print("Enter a valid suffix.")
            continue

        pattern = r"\b\w*" + re.escape(suffix) + r"\b"
        label = "Suffix"
        matches = search_pattern(text, pattern)

    elif choice == "7":
        word = input("Enter word to search: ").strip()

        if not word:
            print("Search word cannot be empty.")
```

```
            continue

        pattern = r"\b" + re.escape(word) + r"\b"
        label = "Word"
        matches = search_pattern(text, pattern)

    else:
        print("Invalid choice. Please select 1 to 8.")
        continue

    print(f"\n{label} Search Results:")

    if matches:
        for match in matches:
            print(match)

        print("Total matches:", len(matches))
    else:
        print("No matches found.")
        print("Total matches: 0")


if __name__ == "__main__":
    main()
```

```
Enter the text (type END on a new line to finish):
Meeting on 12/09/2026
call 9876543210
#NLP
@openAI
natural language processing
END

--- Smart Pattern Matching Engine ---
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
8. Exit
Enter your choice: 2

Phone Number Search Results:
9876543210
Total matches: 1
```

Explanation

- re.findall() extracts every match of a pattern from the text.
- re.IGNORECASE makes the search case-insensitive.
- \b identifies word boundaries, allowing prefix, suffix, and whole-word searches.
- re.escape() treats a user-provided search term as literal text rather than a regular expression.
- (?<!\d) and (?!\d) prevent a 10-digit phone number from being extracted as part of a longer sequence of digits.
- (?<!\w) prevents hashtags and mentions from being matched in the middle of another word.
- The menu supports all seven search types, including word search, and allows repeated searches without re-entering the text.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____